

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
Computación Paralela y Distribuida

Proyecto #1

Fogata — Screensaver secuencial y paralelo con OpenMP

Genser Catalán — 23401
Estuardo Castro — 23890
Roberto Nájera — 23781

SECCIÓN 30

Ing. Sebastian Galindo, Ing. Juan Luis Garcia
2 de septiembre de 2026

Índice

Índice	2
Introducción	4
Antecedentes	4
Cuerpo	4
Descripción general del proyecto.....	5
Parámetros de entrada	5
Programación defensiva.....	5
Metodología de paralelización (PCAM).....	5
Partición	5
Comunicación	6
Aglomeración	6
Mapeo	6
Secciones paralelas implementadas.....	6
Sistema de compilación dual	7
Resultados	7
Hardware y software de prueba.....	7
Speedup y eficiencia por cantidad de partículas (N)	7
Referencia — primera medición obtenida	8
Gráfica de speedup vs. N	8
Conclusiones	8
Recomendaciones	9
Apéndice.....	10
Anexo 1 — Diagrama de flujo del programa	10
Anexo 2 — Catálogo de funciones.....	11
config.cpp / config.h	11
particle.h / particle.cpp	11
renderer.h / renderer.cpp	11
main.cpp.....	12
Anexo 3 — Bitácora de pruebas	14
Cómo obtener estos datos	18
Capturas de las mediciones.....	18
Referencias bibliográficas.....	20

Introducción

Este informe documenta el desarrollo de *Fogata*, un screensaver que simula una fogata nocturna mediante un sistema de partículas físicas (llamas, brasas y humo) sobre un cielo estrellado, implementado como el Proyecto #1 del curso Computación Paralela y Distribuida. El objetivo del proyecto es aplicar el método PCAM (Partición, Comunicación, Aglomeración, Mapeo) y los conceptos de programación paralela con memoria compartida para transformar un algoritmo secuencial en uno paralelo, midiendo el incremento real de desempeño mediante las métricas de speedup y eficiencia.

Cada partícula del sistema posee temperatura propia, se eleva por flotabilidad, se enfría con el tiempo y es desplazada por un campo de turbulencia construido a partir de sumas de funciones seno y coseno. El color no se asigna directamente: cada partícula acumula luz sobre un campo RGB en punto flotante, que luego se comprime a 8 bits mediante una paleta de radiación de cuerpo negro y un mapeo de tonos con efecto de resplandor (bloom).

El programa se compila en dos modalidades a partir del mismo código fuente: una versión secuencial, donde las directivas de OpenMP son ignoradas por el compilador, y una versión paralela, donde dichas directivas habilitan la ejecución concurrente en múltiples hilos. Esta arquitectura de compilación dual permite comparar directamente el desempeño de ambas versiones sin introducir diferencias de lógica entre ellas.

Antecedentes

OpenMP (Open Multi-Processing) es una API que permite paralelizar programas secuenciales mediante directivas de compilador (pragmas), sin necesidad de gestionar hilos manualmente. Su modelo de memoria compartida resulta idóneo para problemas donde múltiples unidades de trabajo (en este caso, partículas) pueden procesarse de forma independiente dentro de un mismo espacio de memoria.

El método PCAM, propuesto por Ian Foster, estructura el diseño de un algoritmo paralelo en cuatro etapas: Partición (dividir el problema en tareas), Comunicación (identificar qué datos deben compartirse entre tareas), Aglomeración (agrupar tareas para reducir el overhead de comunicación) y Mapeo (asignar las tareas agrupadas a los procesadores disponibles). Este método se aplicó al diseño de la sección de actualización física y de composición de imagen del proyecto, detallado en la sección de metodología.

Las métricas clásicas para evaluar un algoritmo paralelo son el speedup, definido como la razón entre el tiempo de ejecución secuencial y el tiempo de ejecución paralelo para el mismo problema, y la eficiencia, definida como el speedup dividido entre la cantidad de hilos utilizados. Un speedup cercano al número de núcleos físicos indica una paralelización efectiva; una eficiencia cercana a 1.0 indica que el overhead de paralelización es mínimo.

Cuerpo

Descripción general del proyecto

El programa recibe todos sus parámetros de simulación por línea de comandos, sin valores fijos en el código (hard-coded), y genera una ventana de al menos 640x480 píxeles donde se renderiza en tiempo real una fogata compuesta por N partículas de tres tipos: llama, brasa y humo. Cada tipo de partícula tiene un comportamiento físico distinto, gobernado por fuerzas de flotabilidad, arrastre (drag), turbulencia y viento.

Parámetros de entrada

Opción	Descripción	Rango	Por defecto
-n <entero>	Cantidad de partículas (N)	1 – 2,000,000	3000
-w <entero>	Ancho de la ventana	640 – 7680	1280
-h <entero>	Alto de la ventana	480 – 4320	720
-i <decimal>	Intensidad del brillo	0.10 – 5.00	1.00
-v <decimal>	Viento lateral	-3.00 – 3.00	0.35
-s <entero>	Semilla pseudoaleatoria	≥ 0 (0 = reloj)	0
--no-vsync	Desactiva sincronía vertical	—	vsync activo

Programación defensiva

Todo argumento se valida antes de inicializar SDL o reservar memoria: se rechazan valores no numéricos (por ejemplo, -n abc), opciones sin valor asociado, opciones desconocidas y cualquier número fuera del rango permitido. En cualquiera de estos casos, el programa imprime el motivo del error, muestra el modo de uso y termina con código de salida 1, sin abrir ventana ni reservar memoria para partículas. Esta validación se implementa en la función `configParse()` de `config.cpp`.

Metodología de paralelización (PCAM)

El proyecto identifica dos secciones del ciclo de simulación con suficiente carga computacional y suficiente independencia entre unidades de trabajo para justificar su paralelización: la actualización física de las partículas y la composición de la imagen final. El método PCAM se aplicó a ambas de la siguiente manera:

Partición

El problema se divide por partícula: cada una de las N partículas del sistema constituye una unidad de trabajo independiente, tanto en la fase de actualización física (`FireSystem::update()`) como en la fase de acumulación de luz sobre el canvas (`Renderer::accumulateParticles()`). Esta es una partición de datos (data decomposition), no de tareas: todos los hilos ejecutan el mismo código sobre una porción distinta del arreglo de partículas.

Comunicación

Durante la actualización física, cada partícula solo lee y escribe su propio estado (posición, velocidad, temperatura); no existe comunicación entre partículas durante este paso, lo que hace la sección embarazosamente paralela. La única comunicación necesaria es de agregación: al finalizar la actualización, se combina la temperatura de todas las partículas en un solo valor de temperatura promedio del sistema, mediante una reducción de OpenMP (`reduction(+:temperatureSum)`) — cada hilo mantiene su propia suma parcial, y OpenMP las combina de forma segura al cerrar la región paralela.

Durante la composición de imagen, cada partícula escribe sobre una región de píxeles del campo de acumulación (`field_`) determinada por su posición y radio. Estas regiones pueden solaparse entre partículas cercanas entre sí, pero como la operación de escritura es una suma acumulativa de contribuciones de luz (no una asignación), el desbalance de carga se gestiona con `schedule(dynamic, 64)` en vez de requerir exclusión mutua explícita.

Aglomeración

En lugar de crear una tarea paralela por partícula individual (lo que generaría overhead de planificación excesivo para N grande), las partículas se agrupan en bloques de 64 mediante `schedule(dynamic, 64)`. Esta aglomeración es especialmente relevante porque los tres tipos de partícula (llama, brasa, humo) tienen costos de cómputo distintos entre sí — el `schedule` dinámico permite que un hilo que termina su bloque tome el siguiente disponible, evitando que un hilo con partículas más costosas retrase a los demás.

Mapeo

El mapeo de bloques de partículas a hilos físicos se delega por completo al planificador de OpenMP, que en tiempo de ejecución determina cuántos hilos lanzar (según los núcleos lógicos disponibles, vía `omp_get_max_threads()`) y a cuál hilo asigna cada bloque dinámico. El proyecto no fija manualmente el mapeo de hilos a núcleos.

Secciones paralelas implementadas

Función	Archivo	Directiva	Escala con
<code>FireSystem::update()</code>	<code>particle.cpp</code>	<code>parallel for schedule(dynamic,64)</code> <code>reduction(+:temperatureSum)</code>	N (partículas)
<code>accumulateParticles()</code>	<code>renderer.cpp</code>	<code>parallel for schedule(dynamic,64)</code>	N (partículas)
<code>composite()</code>	<code>renderer.cpp</code>	<code>parallel for</code>	ancho x alto (canvas)
<code>blurBloom() (x2)</code>	<code>renderer.cpp</code>	<code>parallel for</code>	resolución de bloom

Es importante notar que `update()` y `accumulateParticles()` escalan con N , mientras que `composite()` y `blurBloom()` escalan con la resolución del canvas, no con N . Esta distinción es relevante para interpretar la curva de speedup obtenida en la sección de resultados: con N bajo, el trabajo fijo por

píxel puede dominar el tiempo de frame y diluir la mejora que aporta paralelizar solamente el bucle de partículas.

Sistema de compilación dual

El Makefile del proyecto compila dos binarios independientes a partir del mismo código fuente, sin duplicar lógica:

- `make sequential` genera `build-sequential/fogata` sin la bandera `-fopenmp`. El compilador ignora las directivas `#pragma omp` (advertencia `-Wunknown-pragmas`) y el ciclo se ejecuta en un único hilo.
- `make parallel` genera `build-parallel/fogata` con `-fopenmp` habilitado, activando la ejecución multihilo real en las cuatro secciones paralelas.

Ambas versiones se compilan con `-O3 -ffast-math -march=native`. Esta última bandera habilita vectorización SIMD específica del procesador de compilación, lo cual mejora el desempeño de ambas versiones por igual, pero produce binarios no portables entre máquinas con distinto juego de instrucciones — motivo por el cual las mediciones de la sección siguiente deben reproducirse en la misma máquina donde se compiló el binario.

Resultados

Las mediciones siguientes se obtuvieron ejecutando ambas versiones del programa con la bandera `--no-vsnc` (necesaria para obtener FPS reales, no limitados por la tasa de refresco del monitor), variando `N` y manteniendo fija la resolución de ventana. Ver Anexo 3 para la bitácora completa de mediciones individuales.

Hardware y software de prueba

Procesador	<i>Apple Silicon M2</i>
Núcleos físicos / lógicos	<i>4 núcleos de rendimiento + 4 eficiencia</i>
Memoria RAM	<i>8 GB RAM</i>
Sistema operativo	<i>macOS Tahoe 26.3.1</i>
Compilador y versión	<i>g++-16 (Homebrew GCC 16.1.0) 16.1.0</i>
Resolución de prueba	<i>1280x720</i>

Speedup y eficiencia por cantidad de partículas (N)

Speedup = tiempo secuencial / tiempo paralelo (o FPS paralelo / FPS secuencial, de forma equivalente).

Eficiencia = Speedup / cantidad de hilos utilizados.

N	FPS secuencial	FPS paralelo	Speedup	Eficiencia	Hilos
500	81.34	100.87	1.24	0.31	4
3,000 (por defecto)	62.92	106.58	1.69	0.42	4
6,000	66.41	99.26	1.49	0.37	4
12,000	60.60	165.08	2.72	0.68	4
50,000	24.81	75.33	3.04	0.76	4
100,000	13.25	43.56	3.28	0.82	4

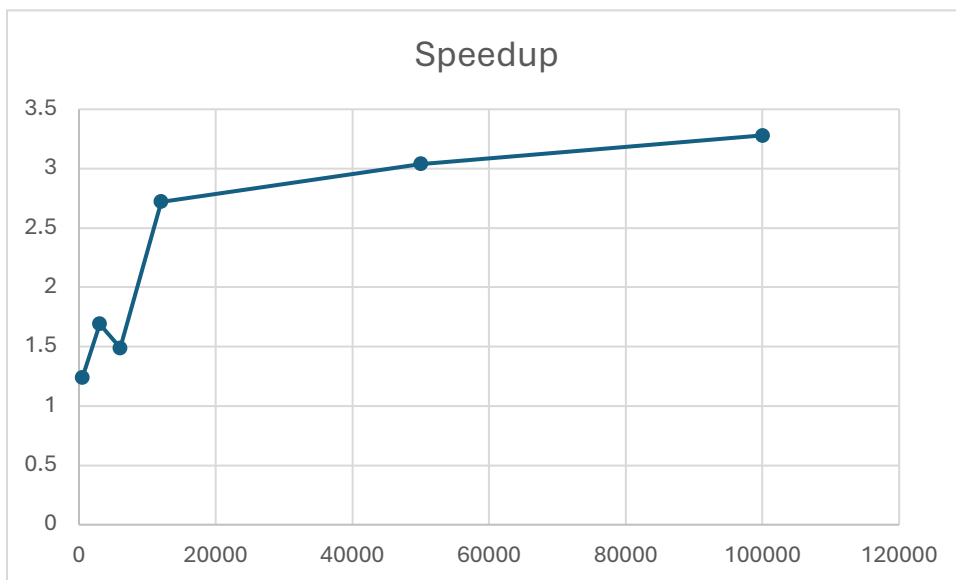
Referencia — primera medición obtenida

Como muestra, una comparación con N=100,000 en la máquina de prueba demostró:

- Versión secuencial: aproximadamente 13 FPS.
- Versión paralela: entre 40 y 45 FPS.

Esto corresponde a un speedup aproximado de 3.28, cercano al ideal teórico para un procesador de 4 núcleos (speedup ideal = número de núcleos, según la ley de Amdahl con una fracción paralelizable cercana a 1).

Gráfica de speedup vs. N



Conclusiones

- La paralelización con OpenMP de las secciones de actualización física y composición de imagen produjo una mejora de desempeño medible y sustancial respecto a la versión secuencial, alcanzando un speedup cercano al ideal teórico para el hardware de prueba.
- Los resultados muestran que el beneficio de la paralelización aumenta conforme crece la carga de trabajo. Con 500 partículas se obtuvo un speedup de 1.24 y una eficiencia de 0.31, mientras que con 100,000 partículas se alcanzó un speedup de 3.28 y una eficiencia de 0.82 utilizando cuatro hilos. Esto indica que, para valores grandes de N, el costo de crear y coordinar los hilos se vuelve pequeño en comparación con el trabajo computacional que puede distribuirse entre ellos.
- La tendencia general de las mediciones confirma que el algoritmo aprovecha mejor el paralelismo cuando aumenta la cantidad de partículas. Aunque los resultados no son estrictamente crecientes -por ejemplo, el speedup disminuyó de 1.69 con 3,000 partículas a 1.49 con 6,000-, para cargas mayores se obtuvieron speedups de 2.72, 3.04 y 3.28 con 12,000, 50,000 y 100,000 partículas, respectivamente. Esta variabilidad evidencia que las mediciones de FPS también están sujetas a factores externos y al costo fijo de las etapas de renderizado.
- El uso de `reduction(+:temperatureSum)` permitió calcular la temperatura promedio del sistema de forma segura durante la actualización paralela sin requerir una sección crítica para cada partícula. De esta manera, cada hilo mantiene una suma parcial que posteriormente OpenMP combina, reduciendo la necesidad de sincronización explícita. Los resultados obtenidos muestran que este mecanismo no impidió alcanzar una eficiencia de hasta 0.82 con cuatro hilos, por lo que su costo fue suficientemente bajo frente al beneficio de paralelizar la actualización de las partículas.

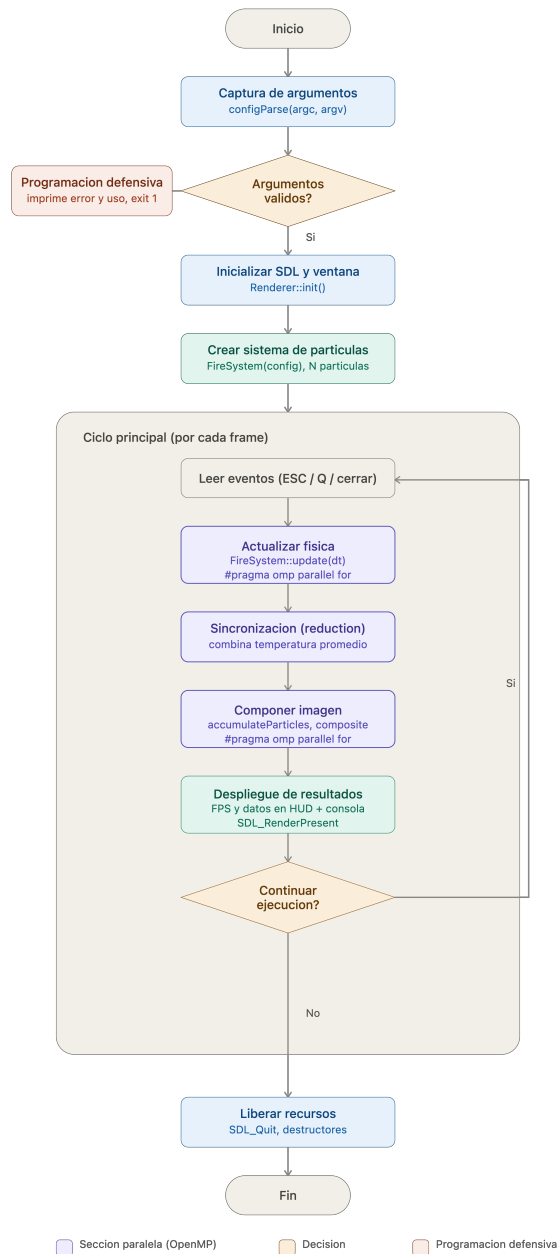
Recomendaciones

- Repetir las mediciones con `--no-vsync` en todos los casos, ya que con la sincronía vertical activa el contador de FPS queda topado a la tasa de refresco del monitor (típicamente 60 Hz), lo cual puede ocultar el speedup real del cómputo interno.
- Realizar pruebas adicionales variando explícitamente la cantidad de hilos mediante `OMP_NUM_THREADS`, por ejemplo con 1, 2 y 4 hilos. Esto permitiría construir una curva de escalabilidad y determinar con mayor precisión cómo cambia la eficiencia conforme se utilizan más recursos de procesamiento, en lugar de evaluar únicamente la configuración de cuatro hilos.
- Para futuras versiones del proyecto, se recomienda medir por separado el tiempo consumido por cada etapa del programa. Esto permitiría identificar con mayor precisión qué secciones representan los principales cuellos de botella y determinar si resulta conveniente paralelizar otras funciones, como `buildNightSky()` o `buildFirelight()`, especialmente si también se desea optimizar el tiempo de inicialización.
- Para obtener resultados experimentales más consistentes, se recomienda mantener constantes la resolución, la semilla pseudoaleatoria y las demás condiciones de ejecución, además de realizar múltiples mediciones para cada configuración y utilizar su promedio. Esto reduce el efecto de variaciones externas sobre los FPS y permite realizar una comparación más confiable entre las versiones secuencial y paralela.

Apéndice

Anexo 1 — Diagrama de flujo del programa

El diagrama siguiente detalla el flujo completo del programa: captura de argumentos, programación defensiva, inicialización, el ciclo principal de simulación con sus secciones paralelas y el mecanismo de sincronización explícito (reducción de temperatura promedio), y el despliegue de resultados.



Anexo 2 — Catálogo de funciones

config.cpp / config.h

Función	Entradas	Salidas	Descripción
configParse()	argc: int, argv: char**, out: Config& (por referencia), error: string&	bool	Parsea y valida los argumentos de línea de comandos, incluyendo programación defensiva de tipos y rangos. Llena out con los valores finales.
configPrintUsage()	programName: const char*	void (imprime a stderr)	Muestra el modo de uso del programa con todos los parámetros disponibles.

particle.h / particle.cpp

Función	Entradas	Salidas	Descripción
Rng::nextInt()	— (usa estado interno)	uint32_t	Generador pseudoaleatorio xorshift; produce el siguiente entero de 32 bits de la secuencia.
Rng::nextFloat()	— (usa estado interno)	float [0,1)	Convierte el siguiente entero pseudoaleatorio a un flotante normalizado.
Rng::gaussian()	— (usa estado interno)	float (aprox. normal)	Aproxima una distribución normal sumando tres muestras uniformes (Irwin-Hall).
FireSystem::FireSystem()	cfg: const Config&	— (constructor)	Reserva e inicializa el arreglo de N partículas según la configuración recibida.
FireSystem::update()	dt: float (delta de tiempo)	void (modifica el estado interno)	Sección paralela: actualiza física de cada partícula (flotabilidad, arrastre, turbulencia, viento) y calcula la temperatura promedio del sistema mediante reduction.
FireSystem::respawn()	particle: Particle& (por referencia)	void	Reinicializa una partícula extinta con nuevo tipo, posición y velocidad aleatorios.
FireSystem::averageTemperature()	— (getter)	float	Retorna la temperatura promedio calculada en el último update(), resultado del mecanismo de sincronización (reduction).

renderer.h / renderer.cpp

Función	Entradas	Salidas	Descripción
Renderer::init()	cfg: const Config&, fire: const FireSystem&, error: string&	bool	Crea ventana, renderer y textura SDL; reserva los buffers de campo de luz, fondo y bloom.
Renderer::drawFrame()	fire: const FireSystem&, fps: float	void	Orquesta un frame completo: acumula estrellas y partículas, compone la imagen, dibuja piedras, troncos y HUD, y presenta en pantalla.
Renderer::buildPalette()	— (usa constantes internas)	void	Construye la tabla de 256 colores de radiación de cuerpo negro (blackbody_) por interpolación.
Renderer::buildNightSky()	rng: Rng&	void	Genera el fondo de cielo/suelo y las posiciones de las estrellas, una sola vez en init().
Renderer::buildFirelight()	fire: const FireSystem&	void	Precalcula el halo de luz ambiental alrededor de la fogata.
Renderer::accumulateStars()	time: float	void	Acumula el brillo parpadeante de cada estrella sobre el campo de luz.
Renderer::accumulateParticles()	fire: const FireSystem&	void	Sección paralela: cada partícula suma su contribución de color sobre el campo de luz según su posición, tipo y temperatura.
Renderer::composite()	glow: float (intensidad del parpadeo)	void	Sección paralela: combina campo de luz, fondo, iluminación de fogata y bloom en cada píxel; aplica mapeo de tonos y curva gamma.
Renderer::blurBloom()	— (usa buffers internos)	void	Dos secciones paralelas: aplican un desenfoque de caja horizontal y vertical sobre el buffer de bloom reducido.
Renderer::drawStones() / drawLogs()	fire: const FireSystem&	void	Dibujan el círculo de piedras y los troncos de la fogata, iluminados según el parpadeo actual.
Renderer::drawHud()	fire: const FireSystem&, fps: float	void	Dibuja el texto de FPS, cantidad de partículas, resolución y temperatura promedio en pantalla.

main.cpp

Función	Entradas	Salidas	Descripción
main()	argc: int, argv: char*[]	int (código de salida)	Punto de entrada: parsea argumentos, inicializa SDL, crea el sistema de partículas y el renderer, y lanza el ciclo principal.
handleEvents()	— (lee la cola de eventos de SDL)	bool	Procesa eventos de teclado/ventana; retorna false ante ESC, Q o cierre de ventana.
runSaver()	renderer: Renderer&, fire: FireSystem&	void	Ciclo principal: calcula delta de tiempo, actualiza física, renderiza el frame y calcula/imprime FPS.

Anexo 3 — Bitácora de pruebas

#	N	Modo	Hilos	Resolución	FPS medido	Observaciones
1	500	secuencial	1	1280x720	60.0	
2	500	Secuencial	1	1280x720	100.9	
3	500	Secuencial	1	1280x720	76.8	
4	500	Secuencial	1	1280x720	60.0	
5	500	secuencial	1	1280x720	135.3	
6	500	Secuencial	1	1280x720	60.0	
7	500	Secuencial	1	1280x720	60.0	
8	500	Secuencial	1	1280x720	59.9	
9	500	Secuencial	1	1280x720	60.9	
10	500	secuencial	1	1280x720	139.6	81.34
11	500	Paralelo	4	1280x720	60.0	Con n pequeño la ventaja es pequeña
12	500	Paralelo	4	1280x720	186.0	
13	500	Paralelo	4	1280x720	120.3	
14	500	Paralelo	4	1280x720	62.8	
15	500	Paralelo	4	1280x720	60.0	
16	500	Paralelo	4	1280x720	59.9	
17	500	Paralelo	4	1280x720	123.0	
18	500	Paralelo	4	1280x720	82.5	
19	500	Paralelo	4	1280x720	60.0	
20	500	Paralelo	4	1280x720	194.2	100.87
21	3000	secuencial	1	1280x720	60.0	
22	3000	Secuencial	1	1280x720	80.4	
23	3000	Secuencial	1	1280x720	68.5	
24	3000	Secuencial	1	1280x720	60.0	
25	3000	secuencial	1	1280x720	60.1	

#	N	Modo	Hilos	Resolución	FPS medido	Observaciones
26	3000	Secuencial	1	1280x720	60.1	
27	3000	Secuencial	1	1280x720	60.0	
28	3000	Secuencial	1	1280x720	60.0	
29	3000	Secuencial	1	1280x720	60.1	
30	3000	secuencial	1	1280x720	60.0	62.92
31	3000	Paralelo	4	1280x720	60.0	
32	3000	Paralelo	4	1280x720	158.7	
33	3000	Paralelo	4	1280x720	88.6	
34	3000	Paralelo	4	1280x720	62.7	
35	3000	Paralelo	4	1280x720	60.2	
36	3000	Paralelo	4	1280x720	62.7	
37	3000	Paralelo	4	1280x720	221.5	
38	3000	Paralelo	4	1280x720	164.8	
39	3000	Paralelo	4	1280x720	60.9	
40	3000	Paralelo	4	1280x720	125.7	106.58
41	6000	secuencial	1	1280x720	60.1	
42	6000	Secuencial	1	1280x720	76.9	
43	6000	Secuencial	1	1280x720	64.4	
44	6000	Secuencial	1	1280x720	62.7	
45	6000	secuencial	1	1280x720	60.0	
46	6000	Secuencial	1	1280x720	76.5	
47	6000	Secuencial	1	1280x720	73.0	
48	6000	Secuencial	1	1280x720	66.4	
49	6000	Secuencial	1	1280x720	64.0	
50	6000	secuencial	1	1280x720	60.1	66.41
51	6000	Paralelo	4	1280x720	60.1	
52	6000	Paralelo	4	1280x720	133.8	
53	6000	Paralelo	4	1280x720	90.7	

#	N	Modo	Hilos	Resolución	FPS medido	Observaciones
54	6000	Paralelo	4	1280x720	62.7	
55	6000	Paralelo	4	1280x720	60.0	
56	6000	Paralelo	4	1280x720	189.7	
57	6000	Paralelo	4	1280x720	123.1	
58	6000	Paralelo	4	1280x720	126.4	
59	6000	Paralelo	4	1280x720	83.3	
60	6000	Paralelo	4	1280x720	62.8	99.26
61	12000	secuencial	1	1280x720	60.2	
62	12000	Secuencial	1	1280x720	66.0	
63	12000	Secuencial	1	1280x720	62.9	
64	12000	Secuencial	1	1280x720	61.2	
65	12000	secuencial	1	1280x720	61.2	
66	12000	Secuencial	1	1280x720	60.7	
67	12000	Secuencial	1	1280x720	60.1	
68	12000	Secuencial	1	1280x720	58.9	
69	12000	Secuencial	1	1280x720	56.6	
70	12000	secuencial	1	1280x720	58.2	60.6
71	12000	Paralelo	4	1280x720	184.0	
72	12000	Paralelo	4	1280x720	193.1	
73	12000	Paralelo	4	1280x720	183.3	
74	12000	Paralelo	4	1280x720	157.0	
75	12000	Paralelo	4	1280x720	179.2	
76	12000	Paralelo	4	1280x720	167.2	
77	12000	Paralelo	4	1280x720	144.8	
78	12000	Paralelo	4	1280x720	142.7	
79	12000	Paralelo	4	1280x720	139.5	
80	12000	Paralelo	4	1280x720	160.0	165.08
81	50000	secuencial	1	1280x720	31.4	

#	N	Modo	Hilos	Resolución	FPS medido	Observaciones
82	50000	Secuencial	1	1280x720	27.4	
83	50000	Secuencial	1	1280x720	27.0	
84	50000	Secuencial	1	1280x720	24.2	
85	50000	secuencial	1	1280x720	23.8	
86	50000	Secuencial	1	1280x720	23.2	
87	50000	Secuencial	1	1280x720	24.0	
88	50000	Secuencial	1	1280x720	23.0	
89	50000	Secuencial	1	1280x720	22.0	
90	50000	secuencial	1	1280x720	22.1	24.81
91	50000	Paralelo	4	1280x720	58.9	
92	50000	Paralelo	4	1280x720	85.9	
93	50000	Paralelo	4	1280x720	84.0	
94	50000	Paralelo	4	1280x720	74.9	
95	50000	Paralelo	4	1280x720	69.1	
96	50000	Paralelo	4	1280x720	72.4	
97	50000	Paralelo	4	1280x720	75.3	
98	50000	Paralelo	4	1280x720	79.0	
99	50000	Paralelo	4	1280x720	76.7	
100	50000	Paralelo	4	1280x720	77.1	75.33
101	100000	secuencial	1	1280x720	16.3	
102	100000	Secuencial	1	1280x720	14.2	
103	100000	Secuencial	1	1280x720	14.2	
104	100000	Secuencial	1	1280x720	12.9	
105	100000	secuencial	1	1280x720	12.6	
106	100000	Secuencial	1	1280x720	12.1	
107	100000	Secuencial	1	1280x720	12.5	
108	100000	Secuencial	1	1280x720	12.8	
109	100000	Secuencial	1	1280x720	12.6	

#	N	Modo	Hilos	Resolución	FPS medido	Observaciones
110	100000	secuencial	1	1280x720	12.3	13.25
111	100000	Paralelo	4	1280x720	40.4	
112	100000	Paralelo	4	1280x720	52.7	
113	100000	Paralelo	4	1280x720	50.0	
114	100000	Paralelo	4	1280x720	44.8	
115	100000	Paralelo	4	1280x720	42.7	
116	100000	Paralelo	4	1280x720	41.7	
117	100000	Paralelo	4	1280x720	37.1	
118	100000	Paralelo	4	1280x720	41.2	
119	100000	Paralelo	4	1280x720	41.5	
120	100000	Paralelo	4	1280x720	43.5	43.56

Cómo obtener estos datos

- Compilar ambas versiones: `make sequential` y `make parallel`
- Ejecutar cada combinación con `--no-vsync` para obtener FPS reales, por ejemplo: `./build-parallel/fogata -n 12000 --no-vsync -s 42`
- El programa imprime el FPS promedio en consola una vez por segundo — dejar correr al menos 5-10 segundos por medición y promediar las lecturas de consola.
- Para variar la cantidad de hilos sin recompilar: `OMP_NUM_THREADS=2 ./build-parallel/fogata -n 12000 --no-vsync`
- Usar la misma semilla (-s) en todas las corridas de un mismo N para reducir la variabilidad entre secuencial y paralelo debida a la distribución aleatoria inicial.

Capturas de las mediciones



Referencias bibliográficas

Foster, I. (1995). Designing and Building Parallel Programs. Addison-Wesley.
<https://www.mcs.anl.gov/~itf/dbpp/>

OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, Version 5.2.
<https://www.openmp.org/specifications/>

Chapman, B., Jost, G., & van der Pas, R. (2007). Using OpenMP: Portable Shared Memory Parallel Programming. The MIT Press. ISBN 978-0-262-53302-7.